

COURSE NOTES

Python Reinforcement Learning using Gymnasium

Agents, environments, Q-learning on Blackjack and Deep Q-Networks on CartPole

Based on the freeCodeCamp full course (about 2 h 38 min) taught in a Google Colab notebook. The notes follow the video in order, using its transcript and the code visible on screen (OCR). Where the spoken transcript was garbled, or the video was imprecise, the notes use standard RL knowledge to correct it and mark those places clearly.

Contents: 4 parts plus advanced topics • 31 sections • full code with line-by-line explanations • glossary and self-test questions

CONTENTS

Table of Contents

1. What is Reinforcement Learning?	3
2. RL Terminology and Real-World Use Cases	4
3. Introducing Gymnasium	4
4. The Gymnasium Workflow and Core Concepts	5
5. Gymnasium API Cheat Sheet	6
6. The Game of Blackjack	7
7. The Blackjack-v1 Environment	8
8. Environment Setup: Installs and Imports	8
9. Taking a Step: Executing an Action	9
10. The Epsilon-Greedy Strategy	10
11. Q-Learning: The Idea Behind the Agent	11
12. Building the BlackjackAgent: The Constructor	12
13. Agent Methods: get_action, update, decay_epsilon	13
14. Hyperparameters and the Training Loop	14
15. Visualising the Training	15
16. Reading the Training Charts	16
17. Other Ways to Solve Blackjack	17
18. The CartPole Environment	18
19. Imports and the PyTorch Modules	19
20. Replay Memory: The Concept	20
21. Replay Memory: The Code	20
22. The DQN Algorithm Step by Step	21
23. The Q-Network Architecture	22
24. Hyperparameters and the AdamW Optimiser	23
25. Networks, Optimiser, Memory and select_action	24
26. Plotting Episode Durations	25
27. The Optimisation Step (optimize_model)	26
28. Loss, Back-Propagation and the Main Training Loop	27
29. CartPole Recap: How the Pieces Fit	28
30. Advanced Topics to Explore Next	29
31. Glossary, Formula Sheet and Self-Test	30

Prerequisites stated in the video: basic Python (loops, classes, methods) and a general idea of what machine learning is. Every RL concept is introduced as it is needed. The video runs everything in **Google Colab**, so no local installation is required, and the instructor links his notebook in the description in case of version problems.

PART A | FOUNDATIONS

1. What is Reinforcement Learning?

Reinforcement learning (RL) is a type of machine learning in which an **agent** learns to make decisions by **interacting with an environment**. There is no labelled data set. Instead the agent tries actions, receives feedback in the form of **rewards** (good) or **penalties** (bad), and gradually learns which actions lead to the best long-term outcome.

The running example: a warehouse robot

The instructor uses a robot that must carry an item from one place to another inside a warehouse. If the robot steps forward and bumps into a wall, we **penalise** it (deduct points). If it steps forward without hitting anything, that is a good action, so we **reward** it (give points). Repeating this over many attempts, the robot learns on its own which steps are safe and eventually reaches the goal.



The loop repeats until the episode ends (goal reached, crash, or time limit).

Agent and environment

- **Agent:** an entity that interacts with an environment in order to learn how to make decisions that maximise a specific goal or objective. It decides autonomously, using the input it receives from the environment, performs actions, and receives rewards or penalties for them.
- **Environment:** the external system or context in which the agent operates. In the robot example it is everything near the robot, i.e. the warehouse. It can be a simulation of a physical system or any other system the agent can act on.
- **Feedback:** the environment responds to each action with a reward or a punishment. The agent uses this feedback to make better decisions next time.

The goal of RL in one sentence

Maximise the **total (cumulative) reward**. The process is trial and error: observe the consequences of an action, adjust behaviour, and over time keep the actions that lead to the highest rewards while avoiding those that lead to penalties.

As training continues the agent **adapts to new situations**, improves its performance and becomes an efficient decision maker, which is exactly what we want from the robot (or any agent).

PART A | FOUNDATIONS

2. RL Terminology and Real-World Use Cases

Vocabulary you will see throughout the course

Term	Meaning	Robot example
Agent	The learner / decision maker	The robot
Environment	The world the agent acts in	The warehouse
State / Observation	What the agent can see about the world right now	Its position / sensor readings
Action	A choice the agent can make	Forward, back, left, right
Reward	Numeric feedback after an action	+ for a safe step, - for hitting a wall
Policy	The rule used to choose actions	"Keep stepping forward unless a wall is ahead"
Episode	One full run from start to a terminal state	Start position to the goal (or a crash)
Terminal state	A state where the episode ends	Goal reached

Where RL is used (examples given in the video)

Domain	What RL does there
Robotics	Grasping objects, navigating, interacting with humans. Example: robots trained to play table tennis that learn to adjust their movement from the position of the ball.
Game playing	Atari games (DeepMind), Blackjack, chess, and AlphaGo, the Google DeepMind program that learned Go at a world-class level.
Autonomous driving	Training self-driving cars to navigate traffic, avoid obstacles and make safe, efficient decisions in complex, dynamic surroundings.
Personalised recommendations	Netflix- or Google-style recommendations: learn what content to recommend to maximise user engagement and satisfaction.

Why this matters for the rest of the notes

Every example above has the same shape: **observe, act, receive reward, repeat**. Gymnasium turns this loop into a few lines of Python, and Q-learning / DQN are two ways of learning from it.

PART B | GYMNASIUM

3. Introducing Gymnasium

Gymnasium is a toolkit (software library) that provides a collection of pre-built environments, or tasks, for testing and developing reinforcement learning algorithms. Its environments are typically game-like, with **well-defined rules and reward structures**, which makes them ideal for evaluating and comparing algorithms. All of them share one **standard interface**, so the same agent code can run on very different tasks. Gymnasium is built for a **single agent**; for several agents in one environment (two taxis, several poker players) the instructor points to the sister library **PettingZoo**, covered at the end. The docs home page shows that only **two lines of code** are needed to set up an environment and interact with it.

Note: where the video is imprecise

The video calls Gymnasium a library "by OpenAI". It is the maintained fork of OpenAI's older *Gym*, now run by the Farama Foundation; install and import gymnasium (not gym).

Environments highlighted in the video

Environment	What it is	Used in this course?
Blackjack	Card game: decide to hit or stick	Yes, Part C (Q-learning)
CartPole	Balance a pole on a moving cart	Yes, Part D (DQN)
Taxi	Grid world: pick up and drop off a passenger	Mentioned
FrozenLake	Walk across a frozen grid avoiding holes	Mentioned
MountainCar	Drive a car up a hill using momentum	Mentioned
CarRacing, BipedalWalker	Racing from images; teaching a two-legged robot to walk	Mentioned
Atari games (e.g. Pong)	Classic arcade games from pixels	Mentioned

PART B | GYMNASIUM

4. The Gymnasium Workflow and Core Concepts

The four steps to start interacting



Applied to the robot: the environment is the warehouse; the policy is "step forward, and if that does not crash into a wall or another object it is a successful step"; we reward good steps and penalise bad ones; then we update the policy so that successful steps are repeated. **This is repeated until performance is satisfactory.**

Main concepts of Gymnasium

Concept	Definition (from the video)	Robot example
Observation space	The set of possible states an agent can observe in the environment	All floor positions / sensor values (the video draws positions 5, 6, 7 ...)
Action space	The set of possible actions the agent can take	Forward, right, left, back
Episode	A complete run through the environment, from the initial state until a terminal state is reached	Start cell to goal cell; training runs many episodes
Wrapper	A tool that modifies an environment's behaviour without changing its code	Add time limits, reward shaping, action masks
Benchmark	Standardised environments used to evaluate and compare RL algorithms	Run two algorithms on the same task and compare scores

- The **action space and observation space are different things**: actions are what the agent *does*; observations are what it *sees*.
- Each episode is a sequence of **state, action, reward** triples. The same robot can start at different positions and end at different terminal locations; each such run is a separate episode.

Discrete vs continuous

The course outline promises the difference between **discrete** and **continuous** spaces. Blackjack has a discrete action space (hit or stick) and a small discrete observation space. CartPole has a discrete action space (left or right) but a **continuous** observation space (four real numbers), which is exactly why a neural network (DQN) is needed there instead of a lookup table.

PART B | GYMNASIUM

5. Gymnasium API Cheat Sheet

These few calls are all you need for every environment in the course. The Blackjack and CartPole sections both use them.

```
import gymnasium as gym

env = gym.make("Blackjack-v1", sab=True, render_mode="rgb_array") # 1. create

obs, info = env.reset() # 2. start a new episode -> first observation

action = env.action_space.sample() # 3. pick an action (here: random)
next_obs, reward, terminated, truncated, info = env.step(action) # 4. act

done = terminated or truncated # 5. episode over? then call env.reset() again
```

What each return value means

Value	Meaning
next_obs	The observation the agent receives after taking the action (the next state).
reward	The reward received for that action.
terminated	True if the environment reached a terminal state, i.e. the game/task actually ended (for Blackjack, a bust or the hand being settled; for CartPole, the pole fell or the cart left the track).
truncated	True if the episode was cut short, usually because a time limit was reached.
info	A dictionary with extra diagnostic information. Ignore it in Blackjack. In Atari environments it holds, for example, a <i>lives</i> key telling how many lives the agent has left.

Note: where the video is imprecise

While explaining `step()` the instructor says it returns "four" variables but then lists five. In Gymnasium it is **five**: observation, reward, terminated, truncated, info. (Old Gym returned four, with a single *done* flag. Gymnasium split it into terminated and truncated.)

Useful methods and attributes

- `env.reset()`: resets the environment to an initial state and returns (*observation*, *info*). Call it at the start of every episode.
- `env.step(action)`: executes one action and changes the state of the environment.
- `env.action_space` and `env.observation_space`: describe valid actions and observations; `env.action_space.sample()` draws a random valid action, and `env.action_space.n` is the number of discrete actions.
- `render_mode="rgb_array"`: makes `env.render()` return an image array, which the video shows with `plt.imshow(frame)` so you can watch the game inside Colab (there is no pop-up window in a notebook).

PART C | SOLVING BLACKJACK WITH Q-LEARNING

6. The Game of Blackjack

The first full project is a Blackjack-playing agent. In the video both sides are played by programs: the agent is the player and the dealer follows fixed rules.

Rules (as listed in the video and the Gymnasium docs)

1. The game uses one or more decks of standard playing cards.
2. The player and the dealer are each dealt **two cards**; one of the dealer's cards is face-down (the observation only shows the face-up card).
3. Card values: **Ace = 1 or 11**; **Jack, Queen, King = 10**; every other card is worth its face value.
4. At each turn the player may **hit** (take another card to improve the hand) or **stick / stand** (keep the current total).
5. The dealer must keep hitting until the hand totals **17 or more**.
6. If the player's total goes over 21 the player **busts** and loses. If the dealer goes over 21, the player wins.
7. If neither busts, the hand with the highest total that is **21 or less** wins.

Note: where the video is imprecise

In the spoken transcript the dealer rule was transcribed as "until 70" and "E reaches over 70". The on-screen rule and the standard game both say the dealer hits until the hand value is **17 or more**.

A usable ace

An ace is **usable** when it can be counted as 11 without pushing the hand over 21. A hand such as Ace + 6 is a "soft 17" (usable ace = true). Take another card and the total might force the ace to count as 1 instead. This is why the observation includes a usable-ace flag: the same total can call for a different decision.

Why Blackjack is a good first RL problem

- The state space is tiny and discrete, so a simple table of values can be learned.
- Episodes are short (one hand), so thousands of games can be played quickly.
- The book **Reinforcement Learning: An Introduction** by Richard Sutton and Andrew Barto contains a Blackjack example, modelled as a **Markov decision process** where every action influences the outcome. It also covers Monte Carlo methods, temporal-difference learning, value functions and policy improvement. The instructor recommends it for further study after this beginner course.

PART C | SOLVING BLACKJACK WITH Q-LEARNING

7. The Blackjack-v1 Environment

The Gymnasium documentation describes Blackjack-v1 as follows.

Item	Specification
Action space	Discrete(2) : 0 = stick (stand), 1 = hit (take another card)
Observation space	A 3-tuple : (1) the player's current sum, (2) the value of the dealer's one showing card (1 to 10, where 1 is an ace), (3) whether the player holds a usable ace (0 or 1)
Observation ranges	Tuple(Discrete(32), Discrete(11), Discrete(2)). Sums above 21 only appear after a bust (the demo's 24); the dealer card is 1 to 10.
Rewards	Win = +1 , Lose = -1 , Draw = 0 . Win with a natural blackjack = +1.5 (if natural=True) or +1 (if natural=False)
Episode end	Player sticks, or the player's sum exceeds 21 (bust)

Constructor arguments

Argument	Meaning
natural	Whether to give an additional reward for starting with a natural blackjack , i.e. an Ace plus a 10-value card (sum of 21). Default False.
sab	Whether to follow the exact rules in the Sutton & Barto book . If True, the natural argument is ignored. The video sets sab=True.
render_mode	Set to "rgb_array" in the video so frames can be drawn with matplotlib during training.

Reading an observation

The first observation printed in the video is (16, 9, False): the player holds **16**, the dealer shows a **9**, and the player has **no usable ace**. The first number is the current hand sum, the second is the dealer's visible card, the third is the usable-ace Boolean.

Why a Q-table fits: the observation space allows at most $32 \times 11 \times 2 = 704$ combinations, so the whole state space is tiny. Combined with two actions, every (state, action) pair can be given its own row in a table. This is exactly what the Q-learning agent built later in this part does.

PART C | SOLVING BLACKJACK WITH Q-LEARNING

8. Environment Setup: Installs and Imports

The notebook starts by installing the libraries and importing everything needed, with a short explanation of each.

```
!pip install gymnasium matplotlib numpy seaborn tqdm

%matplotlib inline          # show plots below the cell (Colab has no pop-up window)

from collections import defaultdict
import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from matplotlib.patches import Patch
from tqdm import tqdm
```


Library	Why it is used here
gymnasium	The environment and the standard RL interface.
numpy	Array manipulation: Q-value arrays, argmax, max, moving averages.
matplotlib	Plotting charts and showing rendered game frames. <code>%matplotlib inline</code> makes plots appear under the cell in Colab.
seaborn	Additional statistical data visualisation (imported with matplotlib).
<code>collections.defaultdict</code>	A dictionary that creates a default value for missing keys, so we never have to check "does this state exist yet?" before reading its Q-values.
<code>matplotlib.patches</code>	Helps draw shapes in plots (Patch).
tqdm	A progress bar: shows training progress as a loader, which is easy to follow.

Note: where the video is imprecise

The instructor mentions installing a specific long-term-support version (a 0.27.x release) of Gymnasium in the video. Newer releases (1.x) changed a few wrapper details; see the version note in the section on reading the training charts. If something fails, use the version from his linked Colab notebook.

Creating and inspecting the environment

```
env = gym.make("Blackjack-v1", sab=True, render_mode="rgb_array")

done = False
observation, info = env.reset()
print(observation)           # e.g. (16, 9, False)
```

- `gym.make` builds the environment by name.
- `env.reset()` starts a new episode: it resets the "board" to an initial state. We use the variable `done = False` because training has just started.
- The printed tuple is *(player sum, dealer card, usable ace)*. The video's run printed **16, 9, False**: no ace at the start, so the Boolean is False.

PART C | SOLVING BLACKJACK WITH Q-LEARNING

9. Taking a Step: Executing an Action

Once the environment has been reset, the agent interacts with it through `env.step(action)`. The action is what the player decides: in Blackjack, take a new card (hit) or keep the current hand (stick). The method executes the action, changes the state of the environment, and returns five values.

```
# sample a random action from all valid actions (0 = stick, 1 = hit)
action = env.action_space.sample()

# execute the action in our environment and receive the infos from the environment
observation, reward, terminated, truncated, info = env.step(action)
```

Reading the sample output in the video

The instructor ran the cell with `action = 1` (hit) and got a new observation whose first number was **24**. The player's hand was now over 21, i.e. a **bust**, with no usable ace. Consequently:

Returned value	Value in the demo	Why
observation	(24, dealer card, False)	Player sum is now 24, no usable ace
reward	-1.0	The player lost the hand
terminated	True	The episode ended (bust)
truncated	False	No time limit was involved
info	{}	Nothing extra for Blackjack-v1

What to do when terminated is True

The current episode is over. Stop acting and call `env.reset()` to begin a new one. This pattern (act, check terminated or truncated, reset) is the skeleton of every RL training loop in this course.

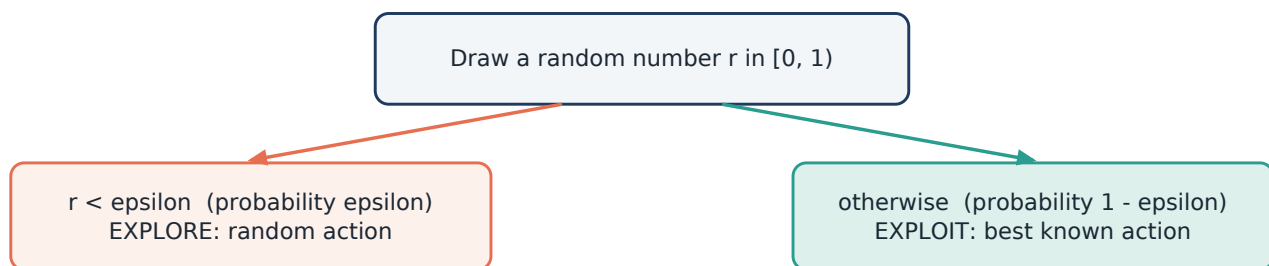
Training means looping this over and over

A single step teaches nothing. Learning happens when this step is repeated across thousands of episodes while an algorithm adjusts its estimates after every step. The next sections build that algorithm: first the action-selection strategy (epsilon-greedy), then the value function (Q-learning).

PART C | SOLVING BLACKJACK WITH Q-LEARNING

10. The Epsilon-Greedy Strategy

The instructor calls this a very effective strategy for Blackjack. At every step the agent chooses between **exploiting** what it already knows and **exploring** something new:



- With probability **1 - epsilon** the agent takes the best action according to its current policy.
- With probability **epsilon** it takes a **random** action.
- This balances **exploitation** of the current best policy against **exploration** of new ones, which can lead to better rewards in the long run.

In the context of Blackjack

- At each step the player decides whether to hit or stick: either the action recommended by the current policy or a random one.
- The policy is learned over time by updating the **action-value estimate of every state-action pair** using the rewards received during the game.
- Early on the agent explores by taking random actions to discover strategies. As the game is played repeatedly it starts to **exploit** the best-known policy, which maximises the expected reward over time.

Epsilon decays

Epsilon starts high (**1.0**, always explore) and is reduced after every episode down to a floor (**0.1**, still explore 10% of the time). The video sets this up with `epsilon_decay = start_epsilon / (n_episodes / 2)`, a linear decrease that reaches the floor halfway through training.

Epsilon	Behaviour
1.0	Pure exploration: every action is random (start of training)
0.5	Half the actions are random, half greedy (about a quarter of the way through)
0.1 (final)	Mostly greedy, with 10% exploration kept so the agent never stops discovering

PART C | SOLVING BLACKJACK WITH Q-LEARNING

11. Q-Learning: The Idea Behind the Agent

Q-learning is a model-free RL algorithm that learns an optimal policy by updating a **Q-value for every state-action pair**. The Q-value function estimates the optimal action to take in each state; the optimal action in a state is the one that **maximises the expected long-term reward**. At decision time the agent picks the action with the highest Q-value for the current state (this is the *argmax* in the code).

The two formulas to remember

```

policy(s) = argmax over a of Q(s, a)           # which action to take

target     = reward + gamma * max over a' of Q(s', a')  # what Q(s, a) should be
TD error   = target - Q(s, a)                        # temporal difference
Q(s, a)    <- Q(s, a) + alpha * TD error              # move a little towards the target

```

The instructor flags the maths as the "complicated" part of the course and says it is fine to take it as given for now: the code computes it for you and the book by Sutton and Barto explains the derivation.

Symbol	Name in the code	Meaning
alpha	learning_rate (0.01)	How far each update moves the old estimate toward the new target. Small = slow, stable.
gamma	discount_factor (0.95)	How much future rewards count compared with immediate reward. Close to 1 = far-sighted.
epsilon	epsilon	Probability of exploring (see previous section).
s, a, s'	obs, action, next_obs	Current state, action taken, state after the action.
terminated	terminated	If the episode ended there is no future, so the future term is 0.

A tiny worked example (numbers are illustrative)

Suppose the agent is in a state where hitting currently has **Q = 0.20**. It hits, the game continues (reward 0), and in the new state the best Q-value is **0.50**. With gamma = 0.95 and alpha = 0.1:

```

future_q_value = 0.50           # not terminated, so we keep the future term
target         = 0 + 0.95 * 0.50 = 0.475
temporal_difference = 0.475 - 0.20 = 0.275
new Q          = 0.20 + 0.1 * 0.275 = 0.2275    # nudged upward

# if instead the hit busted: reward = -1, terminated = True -> future term = 0
target = -1 + 0 = -1 -> TD = -1 - 0.20 = -1.20 -> Q drops

```

Why "temporal difference"?

The TD error is the difference between what the agent predicted (the old Q-value) and what it just observed plus its estimate of the future. The training-error chart in the video simply plots this number, averaged over time.

PART C | SOLVING BLACKJACK WITH Q-LEARNING

12. Building the BlackjackAgent: The Constructor

The agent is a Python class. The constructor takes the learning settings and creates the Q-table.

```
class BlackjackAgent:
    def __init__(self, learning_rate: float, initial_epsilon: float,
                 epsilon_decay: float, final_epsilon: float,
                 discount_factor: float = 0.95):
        """Initialize a Reinforcement Learning agent with an empty dictionary
        of state-action values (q_values), a learning rate and an epsilon."""
        self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))

        self.lr = learning_rate
        self.discount_factor = discount_factor

        self.epsilon = initial_epsilon
        self.epsilon_decay = epsilon_decay
        self.final_epsilon = final_epsilon

        self.training_error = []
```

Line by line

Line	What it does
<code>self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))</code>	The Q-table . Keys are observations (the 3-tuples). The value for each key is an array of length 2 (one Q-value for stick, one for hit). Because it is a <i>defaultdict</i> , an unseen state automatically gets <code>[0, 0]</code> , so there is no need to check whether a key exists. The <code>lambda</code> is needed because <i>defaultdict</i> wants a function that builds the default.
<code>self.lr</code>	Learning rate α : how quickly Q-values change.
<code>self.discount_factor</code>	Gamma, default 0.95: how much we value future rewards.
<code>self.epsilon</code>	Current exploration probability, starting at <code>initial_epsilon</code> .
<code>self.epsilon_decay / final_epsilon</code>	How much to subtract from epsilon each episode, and the minimum it can reach.
<code>self.training_error</code>	A list that collects every TD error so we can plot learning progress later.

- **Arguments:** `learning_rate` (the learning rate), `initial_epsilon` (starting value), `epsilon_decay` (amount to decay by), `final_epsilon` (lowest allowed value) and `discount_factor` (default 0.95).
- The type hints (such as `float`) are for readability. Later the update method uses `tuple[int, int, bool]`; if Python complains about that, add `from __future__ import annotations` at the top, as the instructor had to do.
- The class reads the global `env` to know how many actions exist (`env.action_space.n`, which is 2 for Blackjack).

PART C | SOLVING BLACKJACK WITH Q-LEARNING

13. Agent Methods: get_action, update, decay_epsilon

```
def get_action(self, obs: tuple[int, int, bool]) -> int:
    """Returns the best action with probability (1 - epsilon),
    otherwise a random action with probability epsilon (exploration)."""
    if np.random.random() < self.epsilon:          # explore
        return env.action_space.sample()
    else:                                           # exploit
        return int(np.argmax(self.q_values[obs]))

def update(self, obs: tuple[int, int, bool], action: int, reward: float,
            terminated: bool, next_obs: tuple[int, int, bool]):
    """Updates the Q-value of an action."""
    future_q_value = (not terminated) * np.max(self.q_values[next_obs])
    temporal_difference = (
        reward + self.discount_factor * future_q_value - self.q_values[obs][action]
    )
    self.q_values[obs][action] = (
        self.q_values[obs][action] + self.lr * temporal_difference
    )
    self.training_error.append(temporal_difference)

def decay_epsilon(self):
    self.epsilon = max(self.final_epsilon, self.epsilon - self.epsilon_decay)
```

Method	Role
get_action(obs)	Implements epsilon-greedy. The observation is the tuple (player sum, dealer card, usable ace); the output is an integer (0 stick, 1 hit). A random number below epsilon returns a random sample; otherwise np.argmax over the two Q-values of this state picks the best action.
update(...)	Takes the whole transition: observation, action, reward, terminated flag and next observation. future_q_value is the best Q-value of the next state, multiplied by (<i>not terminated</i>) so it becomes 0 when the episode ended. The TD error is reward + gamma * future - current Q. The Q-value is then moved by lr * TD error and the TD error is recorded for plotting.
decay_epsilon()	Called once per episode. Subtracts epsilon_decay but never goes below final_epsilon (that is what max guarantees).

Note: where the video is imprecise

On screen the video briefly shows small typos while the instructor types (for example `q_values[obs][action]` without `self.` and `[actions]` instead of `[action]`). The listing above is the corrected version. The instructor also pauses on the line using `np.max` to say the formula is "a bit complicated": it is exactly the Q-learning update from the previous section.

Checkpoint

You now have a complete learner: **a table** (`q_values`), **a way to act** (`get_action`), **a way to learn** (`update`) and **a schedule** (`decay_epsilon`). The next step is simply to run it on thousands of episodes.

PART C | SOLVING BLACKJACK WITH Q-LEARNING

14. Hyperparameters and the Training Loop

```
# hyperparameters
learning_rate = 0.01
n_episodes = 100_000
start_epsilon = 1.0
epsilon_decay = start_epsilon / (n_episodes / 2)  # reduce the exploration over time
final_epsilon = 0.1

agent = BlackjackAgent(
    learning_rate=learning_rate,
    initial_epsilon=start_epsilon,
    epsilon_decay=epsilon_decay,
    final_epsilon=final_epsilon,
)
```

Hyperparameter	Value	Comment from the video
learning_rate	0.01	Small steps so noisy Blackjack outcomes do not wreck the table
n_episodes	100,000	Standard for a well-trained agent. For the live render demo the instructor temporarily set it to 10 so the loop finishes quickly, then set it back to 100,000 and let it train.
start_epsilon	1.0	Start by exploring everything
epsilon_decay	start_epsilon / (n_episodes / 2)	Reduces exploration over time, reaching the floor halfway through
final_epsilon	0.1	Where epsilon should be by the end

The training loop

```
from IPython.display import clear_output

env = gym.wrappers.RecordEpisodeStatistics(env, deque_size=n_episodes)

for episode in tqdm(range(n_episodes)):
    obs, info = env.reset()
    done = False
    clear_output()  # keep only the current episode on screen

    # play one episode
    while not done:
        action = agent.get_action(obs)
        next_obs, reward, terminated, truncated, info = env.step(action)

        # update the agent
        agent.update(obs, action, reward, terminated, next_obs)

        frame = env.render()  # show the cards at this moment
        plt.imshow(frame)
        plt.show()

        # update if the environment is done and the current obs
        done = terminated or truncated
        obs = next_obs

    agent.decay_epsilon()
```

- **RecordEpisodeStatistics** is a Gymnasium *wrapper* that records the return and length of every episode (the "statistics" used in the charts).

- **tqdm** shows a progress bar; **clear_output()** wipes the previous frame so you see the game as a live replay.
- **done = terminated or truncated** ends the inner loop: *terminated* means the hand finished (bust or settled), *truncated* means a time limit hit.
- After every episode **decay_epsilon()** makes the agent slightly less random. "Over time it will get smarter."
- Rendering and clearing output on every step is great for a demo ("like watching live Blackjack") but very slow. For a real 100,000-episode run, delete the render lines.

PART C | SOLVING BLACKJACK WITH Q-LEARNING

15. Visualising the Training

After training, the instructor plots three charts to understand how learning went. Raw per-episode values are very noisy, so each series is smoothed with a **rolling (moving) average** of length 500, computed with `np.convolve`.

```
rolling_length = 500
fig, axs = plt.subplots(ncols=3, figsize=(12, 5))

axs[0].set_title("Episode rewards")
reward_moving_average = (
    np.convolve(np.array(env.return_queue).flatten(),
                np.ones(rolling_length), mode="valid") / rolling_length
)
axs[0].plot(range(len(reward_moving_average)), reward_moving_average)

axs[1].set_title("Episode lengths")
length_moving_average = (
    np.convolve(np.array(env.length_queue).flatten(),
                np.ones(rolling_length), mode="same") / rolling_length
)
axs[1].plot(range(len(length_moving_average)), length_moving_average)

axs[2].set_title("Training Error")
training_error_moving_average = (
    np.convolve(np.array(agent.training_error),
                np.ones(rolling_length), mode="same") / rolling_length
)
axs[2].plot(range(len(training_error_moving_average)), training_error_moving_average)

plt.tight_layout()
plt.show()
```

How the moving average works

- `np.ones(rolling_length) / rolling_length` is a window of equal weights that sums to 1, so convolving it with a series gives the **average of each 500 consecutive values**.
- **mode="valid"** only keeps positions where the whole window fits (the curve is 499 points shorter). **mode="same"** keeps the original length but pads the edges with zeros, which makes the curve dip at the start and end.
- The data come from the wrapper (`return_queue`, `length_queue`) and from the agent (`training_error`).

Note: where the video is imprecise

Newer Gymnasium releases changed `RecordEpisodeStatistics`: the `deque_size` argument became `buffer_length` in recent versions. If you get an error on a current install, check the wrapper's documentation or use the exact versions in the instructor's Colab notebook. He also hit several small errors live (misspelled variable names such as `obs` vs `OBS`, a missing import) and fixed them by re-running the notebook with "Run all".

PART C | SOLVING BLACKJACK WITH Q-LEARNING

16. Reading the Training Charts

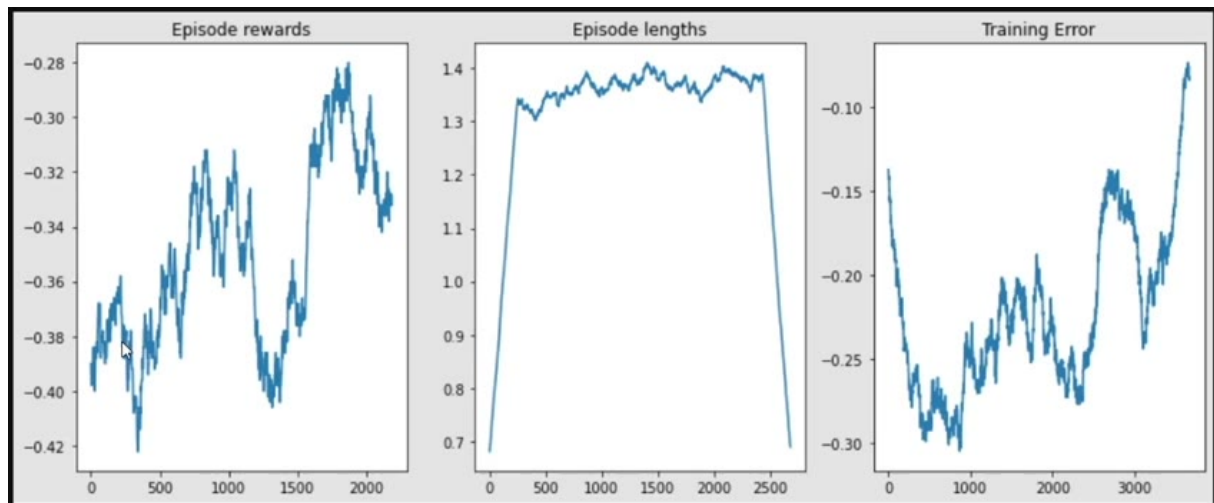


Figure: the three charts produced in the video (frame from the lesson).

Chart	What it shows	What the instructor observed
Episode rewards	Total reward per game (smoothed). One episode is one hand from start to finish; the reward is the net of wins (+1), losses (-1), draws (0) and any bonus.	Very volatile, never a flat line, because Blackjack outcomes are random.
Episode lengths	Number of steps (hit/stick decisions) in each hand.	Between about 1.3 and 1.4 steps per episode: hands are short, rarely more than two decisions.
Training error	The TD error recorded in <code>update()</code> : how wrong the predictions were. The goal of Q-learning is to shrink it as Q-values converge.	Low and improving around 3,000 on the x-axis, then rising again. A persistently high error means the agent is not learning effectively and settings may need adjusting.

How to interpret them (additions to the video)

- **Expected reward stays below zero.** The casino has a small edge, so even a very good Blackjack policy averages slightly under 0 per hand. Success means the curve *trends upward* toward 0, not that it becomes positive. In the figure it climbs from roughly -0.42 to -0.28 during this short run.
- **The dips at both ends of the length chart are not learning.** They come from `mode="same"` zero-padding, as explained in the previous section.
- **Training error is noisy** because rewards are random; look at the trend over many episodes and across different hyperparameters (lower learning rate, more episodes) rather than any single bump.
- **Experiment:** change `n_episodes`, `learning_rate` and `final_epsilon` and see how the three charts change. That is the quickest way to build intuition.

Conclusion of Part C

The Blackjack agent was trained with **Q-learning** and an **epsilon-greedy** policy using only a Python dictionary as its memory. This works because Blackjack has few discrete states. When the state is continuous (like CartPole) a table is impossible, and we turn to a neural network.

PART C | SOLVING BLACKJACK WITH Q-LEARNING

17. Other Ways to Solve Blackjack

The instructor recaps the Part C material and lists alternative methods that you can try on the same environment.

Method	How it works (as described in the video)	Best suited for
Monte Carlo	Model-free. Play many full games, keep track of the rewards obtained for each state-action pair, and update the value function using the average of those rewards.	Episodic tasks like Blackjack
Temporal difference (TD)	Model-free. Update the value function from the difference between the predicted and the actual reward , step by step, without waiting for the episode to end.	Problems where waiting for the end is slow; also continuing tasks
Q-learning	Model-free. Learns the optimal policy by updating the Q-value of each state-action pair; the agent picks the action with the highest Q-value. This is what we implemented.	Finite states and actions (Blackjack)
Deep Q-Network (DQN)	Combines RL with a deep neural network that approximates the Q-values. Next part of this course.	High-dimensional or continuous states, image-based games
Actor-critic	Two networks: the actor chooses actions and the critic evaluates them.	Continuous action spaces

Note: where the video is imprecise

The video describes actor-critic as "model-based". Standard actor-critic algorithms (A2C, A3C, PPO-style) are **model-free**, and TD learning is normally described as suitable for both episodic and continuing tasks. Treat the table as a rough map, then confirm details in the Sutton and Barto book.

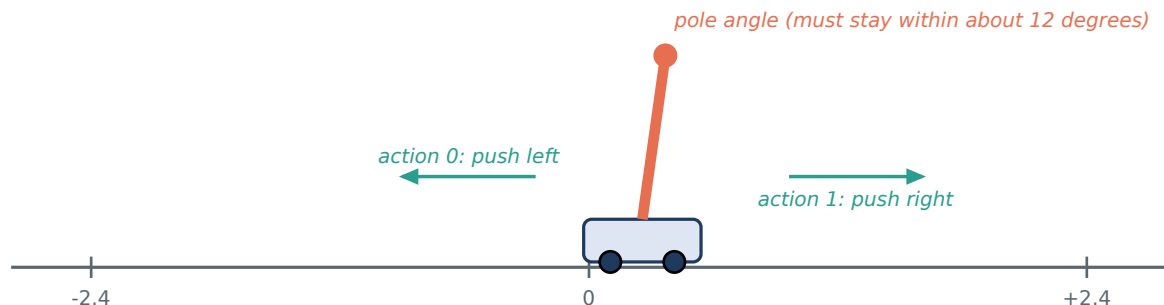
Quick recap of Part C

1. Created the Blackjack-v1 environment with `gym.make`, reset it, and inspected the (sum, dealer card, usable ace) observation.
2. Stepped the environment and understood *next_obs*, *reward*, *terminated*, *truncated*, *info*.
3. Chose epsilon-greedy to balance exploration and exploitation, and decayed epsilon linearly.
4. Built **BlackjackAgent** with a defaultdict Q-table, `get_action`, `update` and `decay_epsilon`.
5. Trained for many episodes with **RecordEpisodeStatistics** and plotted rewards, lengths and training error.

PART D | DEEP Q-NETWORKS ON CARTPOLE

18. The CartPole Environment

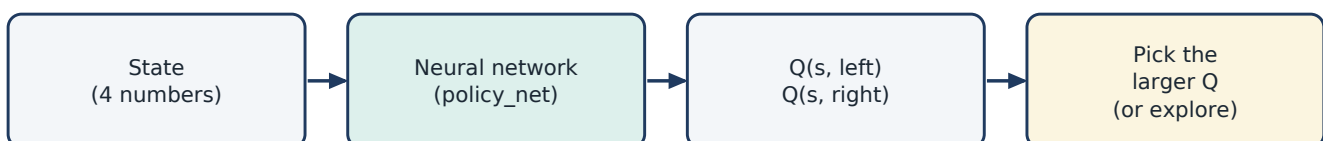
The second project uses **CartPole**, a classic-control task that the instructor compares to an Atari-style game: balance a pole on a cart. The agent must decide between two actions, **move the cart left or right**, so that the pole attached to it stays upright. Random actions (tilting left twice, then right once, and so on) quickly lose the balance, so the agent has to learn.



Item	CartPole-v1
Actions	Discrete(2): push the cart left (0) or right (1)
Observation	4 real numbers (continuous): cart position, cart velocity, pole angle, pole angular velocity. The video passes them to the network without any scaling .
Reward	+1 for every time step the pole stays up
Termination	The pole falls over too far, or the cart moves more than 2.4 units from the centre
Goal	Maximise the number of steps survived. Better policies run longer and accumulate a larger return.

Why a table is no longer enough

In Blackjack the observation was a short tuple from a small discrete set, so every state could have its own row. CartPole's four numbers are **continuous**, so there are infinitely many possible states. Instead of looking values up, we **approximate** the Q-function with a **neural network**: a small fully connected network takes the four inputs and has **two outputs, one Q-value per action**. It is trained to predict the expected value of each action given the state, and the action with the highest value is chosen. This is a **Deep Q-Network (DQN)**.



PART D | DEEP Q-NETWORKS ON CARTPOLE

19. Imports and the PyTorch Modules

```
!pip install "gymnasium[classic_control]" # CartPole lives in the classic-control extras

import gymnasium as gym
import math
import random
import matplotlib
import matplotlib.pyplot as plt
from collections import namedtuple, deque
from itertools import count

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

env = gym.make("CartPole-v1")

# set up matplotlib (inline plots in Colab / Jupyter)
is_ipython = 'inline' in matplotlib.get_backend()
if is_ipython:
    from IPython import display
    plt.ion()

# use the GPU if there is one
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Module / import	Role in the DQN
torch.nn	Tools for building neural networks: fully connected, convolutional and recurrent layers, activation functions and loss functions. Defines our network architecture.
torch.optim	Optimisation algorithms for training: classic stochastic gradient descent and more advanced ones such as Adam and RMSprop. Updates the network weights.
torch.autograd	Automatic differentiation. Computes the gradients of the loss with respect to every parameter during back-propagation so the optimiser can reduce the loss.
namedtuple, deque	Data structures for the replay memory (a named transition and a fixed-length queue).
random, math	Random sampling and the exponential decay of epsilon.
itertools.count	An endless counter used for the steps inside an episode.
matplotlib / IPython.display	Live plotting of episode durations. <code>plt.ion()</code> turns on interactive mode.

Using a GPU

The code detects a GPU with `torch.cuda.is_available()`. In Google Colab open **Runtime, Change runtime type** and choose a GPU to get far more computing power. The notebook is written so it also runs without a GPU (and in a plain Python script): only the number of episodes changes. Tensors and networks are moved to the chosen device with `.to(device)` or `device=device`.

PART D | DEEP Q-NETWORKS ON CARTPOLE

20. Replay Memory: The Concept

Replay memory (experience replay) is a technique used in deep RL to store and manage the agent's experience during training. Each experience is a tuple of four values collected as the agent interacts with the environment:

One transition

(state, action, next_state, reward): "I was in this state, took this action, landed in this next state and received this reward."



When the buffer is full, new experiences overwrite the oldest ones.

Why it helps

- The agent learns from a **random batch of past experiences**, not just the most recent one.
- Random sampling **reduces the correlation** between consecutive experiences (consecutive CartPole frames look almost identical), which improves the stability and convergence of learning. The video notes this has been shown to greatly stabilise and improve DQN training.
- Experiences can be **reused many times** to update the policy and value function, which improves learning efficiency.
- It is implemented as a **fixed-size queue**: when full, the oldest experiences drop out. During training a batch is sampled, the network is updated, and the process repeats until the agent converges.

Mental picture

Instead of learning like someone who only remembers the last five seconds, the agent keeps a diary of recent experiences and studies random pages from it each time.

PART D | DEEP Q-NETWORKS ON CARTPOLE

21. Replay Memory: The Code

Two pieces are written: a **Transition** named tuple that represents a single step, and a **ReplayMemory** class, a cyclic buffer of bounded size with a `sample` method for selecting a random batch.

```

Transition = namedtuple('Transition', ('state', 'action', 'next_state', 'reward'))

class ReplayMemory(object):

    def __init__(self, capacity):
        self.memory = deque([], maxlen=capacity)

    def push(self, *args):
        """Save a transition"""
        self.memory.append(Transition(*args))

    def sample(self, batch_size):
        return random.sample(self.memory, batch_size)

    def __len__(self):
        return len(self.memory)

```

Piece	Explanation
Transition	A named tuple mapping (state, action) pairs to (next_state, reward). The state is the observation of CartPole.
deque(maxlen=capacity)	A double-ended queue with a maximum length. After it is full, appending a new item automatically removes the oldest one (the "overflow" the instructor mentions). Example from the video: capacity 50 and batch size 10.
push(*args)	Takes state, action, next_state, reward, builds a Transition and appends it to the deque.
sample(batch_size)	Returns a random list of batch_size stored transitions.
__len__()	Returns how many transitions are currently stored. The optimiser uses this to check there are enough samples for a batch.

PART D | DEEP Q-NETWORKS ON CARTPOLE

22. The DQN Algorithm Step by Step

DQN is a reinforcement learning algorithm that uses a deep neural network to approximate the Q-function in Q-learning. The steps listed in the video:

1. **Initialise** the Q-network with random weights.
2. **Select an action** with an epsilon-greedy policy (highest Q-value with probability $1 - \epsilon$, random with probability ϵ), as in the Blackjack part.
3. **Execute the action** and observe the next state and reward.
4. **Store** the experience in the replay buffer.
5. **Sample a mini-batch** of experiences from the replay buffer.
6. **Compute the target Q-values** for the mini-batch using the Bellman equation.
7. **Compute the current Q-values** for the mini-batch using the Q-network.
8. **Compute the loss** between the Q-values and the target Q-values.
9. **Update the Q-network** parameters with gradient descent. Repeat until convergence or a fixed number of episodes.

The Bellman target

```

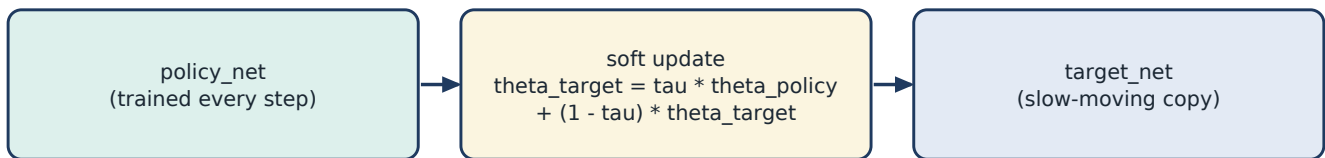
expected Q(s, a) = reward + GAMMA * max over a' of Q_target(s', a')  # 0 future term if terminal
loss              = Huber( Q_policy(s, a), expected Q(s, a) )

```

"Expected Q" is the sum of the immediate reward and the discounted Q-value of the next state; the discount is controlled by gamma. The network is trained so that its prediction for the action actually

taken moves towards this target.

Two networks: policy and target



The **target network** is a copy of the Q-network that is updated less frequently. It is used to compute the target Q-values, with weights that are not updated during the current training iteration. This **stabilises training**: it prevents the Q-values from oscillating and the network from chasing its own moving predictions. In CartPole the algorithm thereby learns to balance the pole by moving the cart left or right; with enough training it can keep the pole up for extended periods.

PART D | DEEP Q-NETWORKS ON CARTPOLE

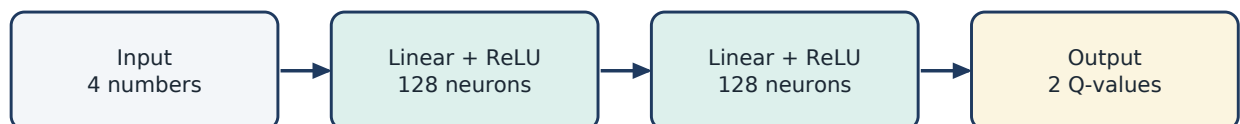
23. The Q-Network Architecture

```

class DQN(nn.Module):

    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()
        self.layer1 = nn.Linear(n_observations, 128)
        self.layer2 = nn.Linear(128, 128)
        self.layer3 = nn.Linear(128, n_actions)

    # Called with either one element to determine next action, or a batch
    # during optimization. Returns tensor([[left0exp, right0exp]...]).
    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        return self.layer3(x)
  
```



- **__init__** takes *n_observations* (size of the state, 4 for CartPole) and *n_actions* (2). It builds **three fully connected layers**: observations to 128, 128 to 128, and 128 to the number of actions.
- **forward(x)** passes the state tensor through layer 1 and a **ReLU** activation, then layer 2 with another ReLU, and finally layer 3 with no activation. The output has one number per action: the predicted Q-value (expected return) of taking that action in this state.
- **Input**: a tensor of size *n_observations* (the state of the environment). **Output**: one neuron per possible action.
- **Why ReLU?** It adds non-linearity so the network can represent more than a straight-line relationship. (The instructor promises to explain it; this is the standard reason.)
- During training the network is updated with the Bellman target to minimise the error between predicted Q-values and target Q-values.

Note: where the video is imprecise

The explanatory text copied from the PyTorch tutorial in the notebook says the model is a "convolutional neural network" that takes the difference between screen patches. That describes the image-based version. The code actually written in the video is a **multi-layer perceptron with three linear layers**, which the instructor confirms ("a three-layer perceptron"), working on the 4 numbers from the environment. Also, the video says "120 neurons" once while typing 128; the code uses 128.

PART D | DEEP Q-NETWORKS ON CARTPOLE

24. Hyperparameters and the AdamW Optimiser

```
# BATCH_SIZE is the number of transitions sampled from the replay buffer
# GAMMA is the discount factor
# EPS_START is the starting value of epsilon
# EPS_END is the final value of epsilon
# EPS_DECAY controls the rate of exponential decay of epsilon, higher means a slower decay
# TAU is the update rate of the target network
# LR is the learning rate of the AdamW optimizer
BATCH_SIZE = 128
GAMMA = 0.99
EPS_START = 0.9
EPS_END = 0.05
EPS_DECAY = 1000
TAU = 0.005
LR = 1e-4
```

Epsilon now decays exponentially

Unlike the linear decay in Blackjack, here epsilon starts at **EPS_START (0.9)** and decays exponentially towards **EPS_END (0.05)** as the step counter grows:

```
eps_threshold = EPS_END + (EPS_START - EPS_END) * math.exp(-1.0 * steps_done / EPS_DECAY)
```

steps_done	0	1,000	2,000	3,000	5,000
epsilon	0.90	0.36	0.17	0.09	0.06

A larger EPS_DECAY gives a slower decay. There is no fixed target: the value keeps changing as steps accumulate, and each step decides whether to act randomly or use the network.

Why AdamW?

- **Adam** (adaptive moment estimation) is a popular optimisation algorithm in deep learning. It is an extension of **stochastic gradient descent**, the most basic algorithm for training neural networks, and combines the advantages of **AdaGrad** and **RMSprop**.
- **AdamW** is a variant that adds **weight-decay regularisation**: a penalty proportional to the size of the weights, which encourages simpler, more generalisable networks and helps prevent overfitting.
- The **learning rate** controls the step size. Too high and the optimiser overshoots and diverges; too low and learning is slow or stuck in a local minimum. Here **LR = 1e-4**.
- The code also passes `amsgrad=True`, a variant of Adam with more stable convergence.

Note: where the video is imprecise

The instructor says "1e-1" once while describing the learning rate; the code and the on-screen constants use **1e-4**. He also mentions "loss function as one minus epsilon" in passing, which is a slip: the loss used is the Huber loss shown in the optimisation section.

PART D | DEEP Q-NETWORKS ON CARTPOLE

25. Networks, Optimiser, Memory and select_action

```
# Get number of actions from gym action space
n_actions = env.action_space.n
# Get the number of state observations
state, info = env.reset()
n_observations = len(state)

policy_net = DQN(n_observations, n_actions).to(device)
target_net = DQN(n_observations, n_actions).to(device)
target_net.load_state_dict(policy_net.state_dict()) # start with identical weights

optimizer = optim.AdamW(policy_net.parameters(), lr=LR, amsgrad=True)
memory = ReplayMemory(10000)

steps_done = 0
```

Variable	Meaning
n_actions	Number of actions from the action space (2).
n_observations	Number of features in the state: reset the environment, take the state, count its length (4).
policy_net / target_net	Two instances of the DQN class on the chosen device. The target net is initialised with the same weights as the policy net.
optimizer	AdamW over the policy network's parameters, with the learning rate set earlier. It optimises the weights.
memory	A ReplayMemory with capacity 10,000: it keeps the most recent 10,000 transitions for training.
steps_done	Counts the actions taken so far; starts at 0 and drives epsilon's decay.

Choosing actions

```
def select_action(state):
    global steps_done
    sample = random.random()
    eps_threshold = EPS_END + (EPS_START - EPS_END) * math.exp(-1. * steps_done / EPS_DECAY)
    steps_done += 1
    if sample > eps_threshold:
        with torch.no_grad():
            # t.max(1) returns the largest column value of each row.
            # The second column of the result is the index where the max was
            # found, so we pick the action with the larger expected reward.
            return policy_net(state).max(1).indices.view(1, 1)
    else:
        return torch.tensor([env.action_space.sample()], device=device, dtype=torch.long)

episode_durations = []
```

- If a random sample is **above** the epsilon threshold, use the network: **exploit**. Otherwise sample a random action: **explore**.
- **torch.no_grad()**: we are only choosing an action, so no gradients need to be tracked.
- **.max(1).indices.view(1, 1)**: for each row (state), find the column with the biggest Q-value; its index is the action (0 or 1); reshape to a 1x1 tensor.
- **episode_durations** records how many steps each episode lasted, to plot progress.

PART D | DEEP Q-NETWORKS ON CARTPOLE

26. Plotting Episode Durations

```
def plot_durations(show_result=False):
    plt.figure(1)
    durations_t = torch.tensor(episode_durations, dtype=torch.float)
    if show_result:
        plt.title('Result')
    else:
        plt.clf()
        plt.title('Training...')
    plt.xlabel('Episode')
    plt.ylabel('Duration')
    plt.plot(durations_t.numpy())
    # Take 100 episode averages and plot them too
    if len(durations_t) >= 100:
        means = durations_t.unfold(0, 100, 1).mean(1).view(-1)
        means = torch.cat((torch.zeros(99), means))
        plt.plot(means.numpy())

    plt.pause(0.001) # pause a bit so that plots are updated
    if is_ipython:
        if not show_result:
            display.display(plt.gcf())
            display.clear_output(wait=True)
        else:
            display.display(plt.gcf())
```

- A helper that plots the **duration of each episode** together with the **average over the last 100 episodes** (the measure used in official evaluations). It updates after every episode, below the cell with the main training loop.
- The episode durations are converted to a tensor and then to a NumPy array for matplotlib. The first plot is a **blue line** of raw durations; the **100-episode moving average** is overlaid as a second line.
- When `show_result` is True the title reads "Result", otherwise "Training..."; axes are labelled Episode and Duration.
- `unfold(0, 100, 1).mean(1)` slides a window of 100 over the series; zeros are padded at the front so the average line lines up with the episode axis.
- `plt.pause(0.001)` lets the figure refresh, and the `is_ipython` branch makes it work in Colab/Jupyter as well as in a plain script (for example VS Code).

How to read the plot

In CartPole a longer episode means the pole stayed up for more steps. A healthy run shows the blue line rising with noise and the average line climbing steadily; flat or collapsing averages suggest a problem with hyperparameters.

PART D | DEEP Q-NETWORKS ON CARTPOLE

27. The Optimisation Step (optimize_model)

This is the heart of DQN. It looks intimidating but does a simple thing: **sample a mini-batch, compute predictions and targets, measure the error, and nudge the network.**

```
def optimize_model():
    if len(memory) < BATCH_SIZE:          # not enough samples yet -> do nothing
        return
    transitions = memory.sample(BATCH_SIZE)
    # Transpose the batch: list of Transitions -> Transition of lists
    batch = Transition(*zip(*transitions))

    # mask of non-final states (final = episode ended, next_state is None)
    non_final_mask = torch.tensor(tuple(map(lambda s: s is not None, batch.next_state)),
                                   device=device, dtype=torch.bool)
    non_final_next_states = torch.cat([s for s in batch.next_state if s is not None])
    state_batch = torch.cat(batch.state)
    action_batch = torch.cat(batch.action)
    reward_batch = torch.cat(batch.reward)

    # Q(s_t, a): the model computes Q for both actions; gather picks the action taken
    state_action_values = policy_net(state_batch).gather(1, action_batch)

    # V(s_{t+1}) = max_a Q_target(s_{t+1}, a); stays 0 for final states
    next_state_values = torch.zeros(BATCH_SIZE, device=device)
    with torch.no_grad():
        next_state_values[non_final_mask] = target_net(non_final_next_states).max(1).values
    # expected Q values = discounted future value + reward
    expected_state_action_values = (next_state_values * GAMMA) + reward_batch
```

Step	Why
Return early if <code>len(memory) < BATCH_SIZE</code>	We need enough stored transitions to fill a mini-batch.
<code>Transition(*zip(*transitions))</code>	Converts a batch-array of transitions into a transition of batch-arrays (all states together, all actions together, ...). It looks odd but is just a transpose.
<code>non_final_mask</code>	A Boolean mask of the states that are not final. When the pole fell, <code>next_state</code> was stored as <code>None</code> , and there is no future value to compute.
<code>policy_net(state_batch).gather(1, action_batch)</code>	The network outputs Q for both actions; gather keeps only the Q-value of the action that was actually taken.
<code>target_net(...).max(1).values</code>	The best Q-value of the next state according to the (frozen) target network. Final states keep 0.
<code>expected = next_values * GAMMA + reward</code>	The Bellman target: immediate reward plus discounted future value.

PART D | DEEP Q-NETWORKS ON CARTPOLE

28. Loss, Back-Propagation and the Main Training Loop

```
# Compute Huber loss
criterion = nn.SmoothL1Loss()
loss = criterion(state_action_values, expected_state_action_values.unsqueeze(1))

# Optimize the model
optimizer.zero_grad()
loss.backward()
# In-place gradient clipping
torch.nn.utils.clip_grad_value_(policy_net.parameters(), 100)
optimizer.step()
```

- **Huber loss (SmoothL1Loss)** is a smooth approximation of the mean-squared-error loss that is less sensitive to outliers.
- **zero_grad(), backward(), step():** clear old gradients, back-propagate the loss through the policy network (autograd), then let AdamW update the weights.
- **clip_grad_value(..., 100)** clips gradients to a maximum value to prevent the **exploding-gradient** problem.

The main loop

```
if torch.cuda.is_available():
    num_episodes = 600
else:
    num_episodes = 50

for i_episode in range(num_episodes):
    state, info = env.reset()
    state = torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0)
    for t in count():
        action = select_action(state)
        observation, reward, terminated, truncated, _ = env.step(action.item())
        reward = torch.tensor([reward], device=device)
        done = terminated or truncated

        if terminated:
            next_state = None
        else:
            next_state = torch.tensor(observation, dtype=torch.float32, device=device).unsqueeze(0)

        memory.push(state, action, next_state, reward)  # store the transition
        state = next_state                             # move to the next state

        optimize_model()                               # one step on the policy network

        # soft update of the target network: theta' = tau*theta + (1 - tau)*theta'
        target_net_state_dict = target_net.state_dict()
        policy_net_state_dict = policy_net.state_dict()
        for key in policy_net_state_dict:
            target_net_state_dict[key] = (policy_net_state_dict[key] * TAU
                                          + target_net_state_dict[key] * (1 - TAU))
        target_net.load_state_dict(target_net_state_dict)

    if done:
        episode_durations.append(t + 1)
        plot_durations()
        break

print('Complete')
plot_durations(show_result=True)
```

```
plt.ioff()
plt.show()
```

Note: where the video is imprecise

On the notebook screen the soft-update block briefly shows `policy_net.state_dict = target_net.state_dict()`. That would blend the target network with itself and the policy would never influence it. The correct line, used above, reads from **policy_net**. The instructor says 600 episodes on a GPU (50 otherwise); the on-screen cell is edited around 500-600, so feel free to change it and compare results, which he encourages.

PART D | DEEP Q-NETWORKS ON CARTPOLE

29. CartPole Recap: How the Pieces Fit

Component	Job	Where in the code
Environment	Simulates the cart and pole, returns 4 numbers and +1 reward per step	<code>gym.make("CartPole-v1")</code>
Policy network	Predicts Q-values for left/right from the state	DQN class, <code>policy_net</code>
Epsilon-greedy	Explore early (random), exploit later (network)	<code>select_action</code> , <code>EPS_*</code> constants
Replay memory	Stores transitions; supplies random mini-batches	<code>Transition</code> , <code>ReplayMemory</code>
Target network	Provides stable targets; updated slowly with TAU	<code>target_net</code> , soft update in the main loop
Loss + optimiser	Huber loss, AdamW, gradient clipping	<code>optimize_model</code>
Monitoring	Episode durations and a 100-episode average	<code>plot_durations</code>

Episode versus iteration

An **episode** is one sequence from a start to an end: in CartPole it starts at the initial state and ends when the pole falls, the cart drifts beyond 2.4 units, or the maximum number of steps is reached. An **iteration** (a step of the inner loop) is a single action inside that episode. With a GPU the notebook plays the game about 600 times, i.e. 600 episodes.

Blackjack (Q-table) versus CartPole (DQN)

	Blackjack Q-learning	CartPole DQN
State	Small discrete tuple	Four continuous numbers
Value storage	defaultdict Q-table	Neural network weights
Exploration	Linear epsilon decay (1.0 to 0.1)	Exponential decay (0.9 to 0.05)
Extra tricks	None	Replay memory, target network, gradient clipping
Update	$Q += \alpha * \text{TD error}$	Gradient descent on Huber loss (AdamW)
Progress plot	Rewards, lengths, training error	Episode durations and 100-episode mean

Conclusion of Part D

You can now explain every line of a working DQN: what is stored, how actions are chosen, how targets are built, and how the networks are updated. The instructor encourages changing the number of episodes and the hyperparameters and comparing the duration plots.

BEYOND THE COURSE

30. Advanced Topics to Explore Next

The final section of the video lists directions to explore once the basics are comfortable.

Topic	Description (from the video)	Where to go
Deep reinforcement learning	Deep learning plus RL, using neural networks to approximate Q-values. Algorithms such as DQN and actor-critic have solved complex tasks in Atari games, robotics and autonomous driving.	DQN variants, actor-critic
Policy gradients	Optimise the policy function directly by adjusting its parameters along the gradient, without computing Q-values. Advantage: they can handle continuous action spaces , unlike Q-learning.	REINFORCE, PPO
Multi-agent RL	Several agents interact in a shared environment, each learning a policy (for example two cars in one environment, or several poker players). Use the sister library PettingZoo .	PettingZoo docs
Imitation learning	Learning from demonstration: train an agent to mimic an expert, used in driving, robotics and games. MuJoCo humanoid locomotion tasks are given as examples.	Gymnasium MuJoCo envs
Transfer learning	Reuse knowledge learned in one task to speed up learning on another and improve performance.	Pretrained policies

Other items from the course outline

- **Stable-Baselines (OpenAI Baselines style libraries)** for benchmarking and running ready-made RL algorithms.
- **Wrappers** to modify environment dynamics (time limits, reward shaping) without changing the environment code.
- **Combining environments** to create more complex scenarios (two taxis, multiple players in a poker environment).
- **Integration with TensorFlow and PyTorch.** (This course already used PyTorch for DQN.)

PettingZoo, as shown at the end of the video

The instructor opens the PettingZoo documentation to show multi-agent games: **Mario Bros** (Mario and Luigi as two agents with a defined action space), **Space Invaders**, and a **classic games** section containing **chess** and other board games. His advice: master the basics here, then read the PettingZoo docs, create your own environments and agents, and apply them to real-world problems.

Note: where the video is imprecise

Where the video says Gymnasium itself provides multi-agent, imitation-learning or transfer-learning environments, the multi-agent ones actually live in **PettingZoo**, and imitation/transfer-learning experiments typically use datasets or MuJoCo/custom environments with extra libraries. Use the video's list as a menu of topics, not as a list of built-in Gymnasium features.

REVISION

31. Glossary, Formula Sheet and Self-Test

Glossary

Term	Short definition
Agent / Environment	The learner / the world it acts in
Policy	The rule that maps states to actions
Observation space / Action space	What the agent can see / what it can do
Episode	One run from a start state to a terminal state
terminated vs truncated	The task truly ended vs it was cut short (e.g. time limit)
Wrapper	Modifies an environment's behaviour without changing its code
Exploration / Exploitation	Try something new / use what you know works
Q-value	Expected long-term reward of taking an action in a state
Temporal difference	Target minus current estimate; the learning signal
Discount factor (gamma)	Weight of future rewards compared with immediate ones
Replay memory	Fixed-size buffer of past transitions sampled at random
Target network	Slowly-updated copy of the Q-network that gives stable targets
Soft update (TAU)	Blend a small fraction of policy weights into the target weights each step

Formulas used in the course

```

epsilon-greedy      a = random action if rand < eps else argmax_a Q(s, a)
Q-learning update  Q(s,a) += alpha * ( r + gamma * (1 - terminated) * max_a' Q(s',a') - Q(s,a) )
epsilon (Blackjack) eps = max(final_eps, eps - eps_decay) # linear
epsilon (DQN)      eps = EPS_END + (EPS_START - EPS_END) * exp(-steps / EPS_DECAY)
DQN target         y = r + GAMMA * max_a' Q_target(s',a') # y = r if terminal
soft update        theta_target = TAU * theta_policy + (1 - TAU) * theta_target

```

Self-test (try to answer before reading the answers below)

1. What is the difference between the observation space and the action space?
2. In Blackjack-v1, what do the three numbers of an observation mean, and what do actions 0 and 1 do?
3. When should you call `env.reset()`?
4. Why does epsilon start at 1.0 and finish at 0.1 in the Blackjack agent?
5. Why is a `defaultdict` used for the Q-table?
6. Why is the future Q-value multiplied by (not terminated)?
7. Why is Q-learning with a table unsuitable for CartPole?
8. What two things does the replay memory improve about learning?
9. What is the purpose of the target network, and what does TAU control?
10. Which loss does `optimize_model` use, and why clip gradients?

Answers. (1) Observations are what the agent sees; actions are what it can do. (2) Player sum, dealer's visible card, usable ace; 0 = stick, 1 = hit. (3) At the start of each episode and whenever terminated or truncated is true. (4) Explore everything early, then mostly exploit while keeping 10% exploration. (5) Unseen states get a zero array automatically. (6) A finished episode has no future reward. (7) CartPole's state is continuous, so there are infinitely many states. (8) It decorrelates samples (stability) and lets experiences be reused (efficiency). (9) It gives stable Bellman targets; TAU is the rate at which it follows the policy network. (10) Huber (SmoothL1) loss; clipping prevents exploding gradients.